MODUL 2

KOMUNIKASI WEBSOCKET DENGAN SERVER LOKAL

TUJUAN

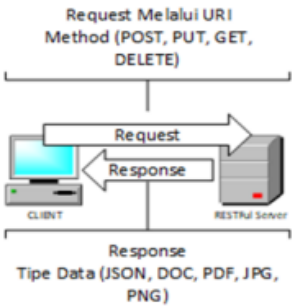
1) Mengetahui protokol HTTP (Hypertext Transfer Protocol).
2) Memahami komunikasi protokol HTTP.
3) Memahami penggunaan WebSocket dalam konteks sistem IoT.

REQUIREMENT

1) Arduino IDE/PlatformIO.
2) ESP32/ESP8266.
3) Server Lokal (Raspberry Pi 3B).
4) Sensor (opsional).
5) Aktuator (opsional).

2.1 Hypertext Transfer Protocol (HTTP)

HTTP (Hypertext Transfer Protocol) adalah protokol lapisan aplikasi untuk pertukaran representasi sumber daya antara client dan server. Client mengirimkan request yang berisi metode, target, header, dan bila diperlukan body. Server kemudian mengirimkan response yang berisi status, header, dan representasi data. HTTP dapat digunakan untuk dokumen, data terstruktur, gambar, video, maupun layanan API, baik melalui jaringan lokal maupun internet. Browser web merupakan salah satu client HTTP, tetapi HTTP juga digunakan oleh aplikasi seluler, perangkat IoT, dan layanan antarsistem.



Gambar 2.1: Visualisasi HTTP

2.1.1 CRUD

CRUD adalah singkatan dari Create, Read, Update, dan Delete. Istilah ini menjelaskan empat operasi dasar terhadap data dan dapat diterapkan pada berbagai jenis penyimpanan, tidak terbatas pada basis data relasional. Pada sistem berbasis HTTP, operasi CRUD sering dipetakan ke metode HTTP, tetapi semantik metode HTTP tetap mengikuti spesifikasi protokol dan tidak selalu sama persis dengan operasi basis data.

Fungsi-fungsi dalam CRUD adalah sebagai berikut.

- **CREATE :** Untuk membuat entri atau record baru dalam sebuah tabel di dalam database. Di dalam aplikasi berbasis SQL, fungsi create sering kali disebut dengan insert
- **READ :** Untuk membaca entri data yang sudah terdaftar dalam sebuah database. Namun, fungsi ini tidak akan memberikan akses untuk mengubah data yang telah tersimpan di dalam database.
- **UPDATE :** Untuk memperbarui entri data ketika informasi di dalamnya memerlukan perubahan. Dengan fungsi ini, Anda dapat mengubah detail dari sebuah entri di database.
- **DELETE :** Untuk menghapus entri data yang sudah tidak diperlukan dalam sebuah database. Ketika menggunakan fungsi ini, Anda akan mengakses detail terkait sebuah entri dan kemudian memberikan perintah kepada sistem untuk menghilangkannya dari database.

Terdapat beberapa variasi istilah yang dikembangkan dari konsep CRUD. Istilah-istilah berikut bukan bagian dari standar HTTP, melainkan pola penamaan operasi aplikasi:

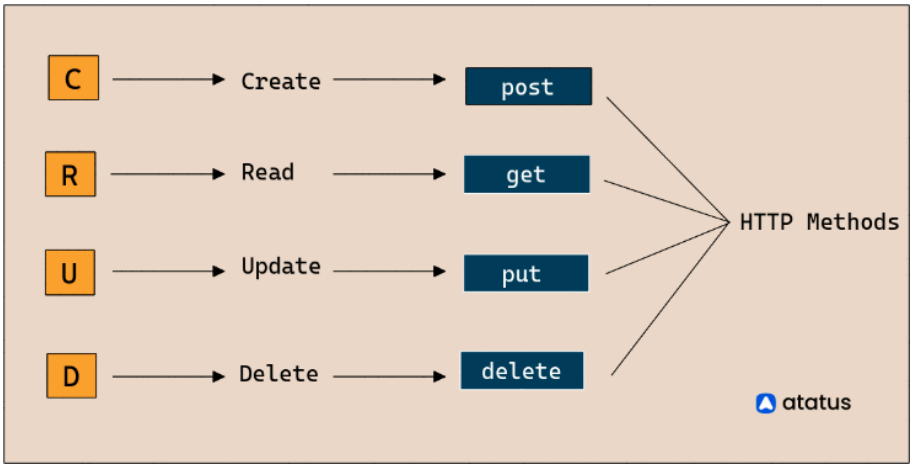
- BREAD (Browse, Read, Edit, Add, Delete)
- CRAP (Create, Replicate, Append, dan Process)
- CRUDL (Create, Read, Update, Delete, List)
- DAVE (Delete, Add View, Edit)

2.1.2 Metode Komunikasi Protokol HTTP

Metode HTTP memiliki semantik tersendiri dan dapat digunakan untuk mendukung operasi CRUD pada API:

- **GET** meminta representasi sumber daya dan tidak seharusnya mengubah keadaan server.
- **POST** mengirimkan data untuk diproses sesuai fungsi sumber daya; metode ini sering digunakan untuk membuat entri baru.
- **PUT** membuat atau mengganti representasi sumber daya pada target yang ditentukan.

- **PATCH** menerapkan perubahan sebagian terhadap sumber daya.
- **DELETE** meminta penghapusan hubungan sumber daya pada target yang ditentukan.



Gambar 2.2: Metode Komunikasi HTTP

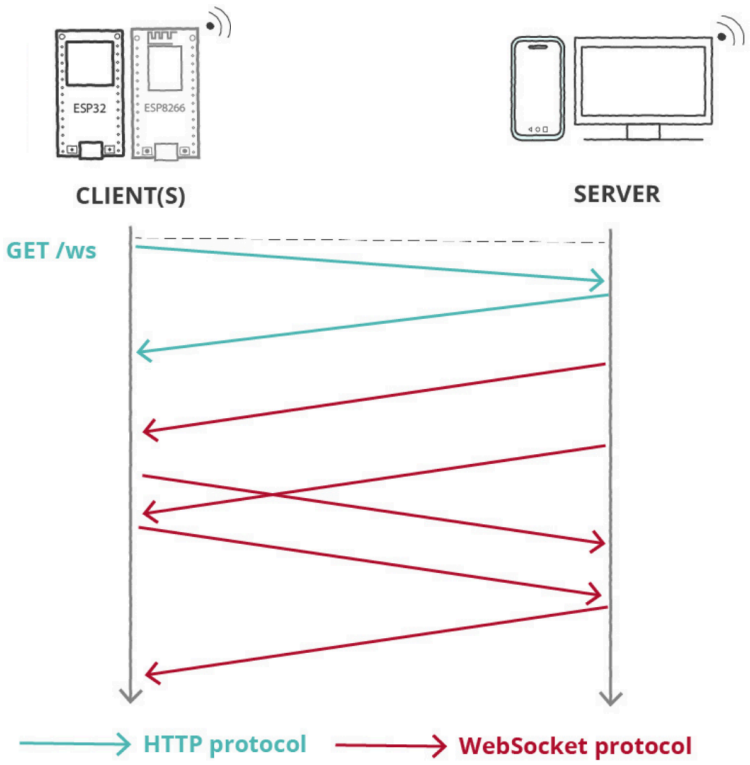
Selain dari metode yang telah disebutkan sebelumnya, ada beberapa metode komunikasi yang dapat digunakan pada protokol HTTP seperti yang ditunjukkan pada tabel berikut.

Tabel 2.1: Jenis dan Fungsi Tiap Metode HTTP

Metode HTTP	Definisi	Penjelasan
GET	Mengambil Sumber daya	Digunakan untuk mengambil data tertentu dari server
POST	Memproses data	Digunakan untuk mengirim data agar diproses sesuai fungsi sumber daya
PUT	Membuat atau mengganti sumber daya	Digunakan untuk membuat atau mengganti representasi sumber daya pada target
DELETE	Menghapus sumber daya	Digunakan untuk menghapus data tertentu dari server
PATCH	Mengubah bagian sumber daya	Digunakan untuk memperbarui atau mengubah bagian dari data yang sudah ada pada server
HEAD	Mendapatkan informasi header	Digunakan untuk meminta informasi header dari data yang diminta melalui metode GET
OPTIONS	Mendapatkan informasi dari server	Digunakan untuk meminta informasi tentang metode-metode yang didukung oleh server untuk data tertentu
CONNECT	Membentuk tunnel	Digunakan untuk meminta proxy membentuk tunnel ke server tujuan
TRACE	Melakukan diagnostik	Digunakan untuk meminta penerima akhir mengirimkan kembali request yang diterima

2.2 WebSocket

WebSocket adalah protokol yang menyediakan komunikasi dua arah melalui satu koneksi TCP. Koneksi diawali dengan HTTP Upgrade handshake, kemudian client dan server dapat saling mengirim frame tanpa membuat request HTTP baru untuk setiap pesan. WebSocket sesuai untuk aplikasi yang memerlukan pertukaran data interaktif atau berfrekuensi tinggi, tetapi latensi aktual tetap dipengaruhi oleh jaringan, implementasi, dan beban server.



Gambar 2.3: Alur Komunikasi WebSocket

Berikut adalah alur komunikasi WebSocket:

a. Client mengirimkan request HTTP khusus untuk meminta peningkatan koneksi menjadi WebSocket. Berikut contoh header request:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://example.com
```

Ketika server menerima permintaan tersebut, maka server akan merespons dengan:

```
HTTP/1.1 101 Switching Protocols
Upgrade: WebSocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

b. Setelah handshake berhasil, client dan server dapat saling mengirim frame. Frame WebSocket tetap memiliki header protokol, tetapi tidak membawa seluruh header HTTP pada setiap pesan. Struktur frame memuat unsur berikut:
   - **FIN bit** : Indikator apakah frame tersebut merupakan akhir dari pesan.
   - **Opcode** : Tipe dari frame (text, binary, close, ping, pong).

- **Mask bit** : Menunjukkan apakah payload diberi masking. Frame dari client ke server wajib dimasking, sedangkan frame dari server ke client tidak dimasking. Masking bukan enkripsi dan tidak memberikan kerahasiaan.
- **Payload length** : Ukuran dari data pesan yang dikirim
- **Payload data** : Konten data yang dikirim

Berikut adalah perbedaan antara komunikasi WebSocket dengan komunikasi HTTP tradisional:

Tabel 2.2: Perbedaan WebSocket dengan HTTP Tradisional

Perbedaan	HTTP	WebSocket
Pola Komunikasi	Menggunakan pola request-response; umumnya client memulai pertukaran	Mempertahankan koneksi setelah handshake sehingga client dan server dapat saling mengirim pesan
Overhead Koneksi	Setiap request dan response membawa header HTTP; ukurannya bergantung pada konfigurasi	Setelah handshake, overhead frame bervariasi menurut panjang payload, masking, dan ekstensi
Latensi	Memerlukan komunikasi bolak-balik di setiap interaksinya	Mengurangi kebutuhan handshake atau request baru untuk setiap pesan, tetapi tidak menghilangkan latensi jaringan
Skalabilitas	Oleh karena sifatnya stateless, maka skalabilitas secara horizontal menjadi lebih mudah	Koneksi WebSocket sifatnya stateful, sehingga lebih kompleks dari segi skalabilitas

2.3 Implementasi Protokol WebSocket

2.3.1 Server Python Raspberry Pi

Sebelum melakukan komunikasi, kita perlu membuat server WebSocket agar ESP32 dapat terhubung dan bertukar pesan. Buatlah folder baru untuk menyimpan kode server yang akan dipakai, misal dengan nama `server-websocket`. Instal paket Python yang diperlukan terlebih dahulu jika belum terinstal:

Instalasi paket python dan virtual environment

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Berikutnya, buatlah virtual environment di dalam folder yang sudah dibuat sebagai berikut:

Membuat virtual environment python

```
python3 -m venv .venv

# Mengaktifkan virtual environment
source .venv/bin/activate
```

Setelah masuk ke dalam virtual environment, instal paket `websockets` pada Python sebagai berikut di terminal:

Instalasi paket WebSocket Python

```
pip install websockets
```

Buatlah kode sumber python di dalam folder tersebut, misal dengan nama `main.py`. Isi dengan konten berikut:

Import library dan konfigurasi variabel

```python
import asyncio  # Mendukung eksekusi coroutine secara asinkron

# Pustaka WebSocket
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0"  # Mendengarkan pada semua antarmuka jaringan
PORT: int = 8765
```

Fungsi callback untuk menerima beberapa client

```python
async def handle_client(websocket: ServerConnection) -> None:
    # Di sini, kita akan membedakan tipe pesan yang dikirim pada websocket
    # Untuk path yang akan dipakai saat ini adalah "/esp32"
    path: str = websocket.request.path

    # Kita dapat atur path yang kita ingin proses
    if path == "/esp32":
        print("ESP32 device connected")
        try:
            async for message in websocket:
                # Message from ESP32
                print( f"ESP32 message: {message}" )

                # Reply
                response: str = ( f"ACK from Raspberry Pi: {message}" )
                await websocket.send(response)

        except ConnectionClosed:
            print("ESP32 disconnected")
    # Anda dapat tambah path-path yang lain sesuai kebutuhan Anda
    # Case default ketika tidak sesuai
    else:
        print(f"Unknown path: {path}")
```

Pada kode di atas, kata kunci `async` mendefinisikan coroutine yang dapat dijalankan secara asinkron atau dalam artian lain tidak berurutan. Objek `websocket` mewakili satu koneksi client. Pesan diterima melalui perulangan `async for`, sedangkan metode `websocket.send()` digunakan untuk mengirim balasan pada koneksi tersebut.

Fungsi main pada Python

```python
async def main() -> None:
    # Isi dengan konfigurasi yang telah kita buat
    async with serve(handle_client, HOST, PORT):
        print(f"WebSocket server running " f"on {HOST}:{PORT}")
        # Menunggu pesan...
        await asyncio.Future()

if __name__ == "__main__":
    asyncio.run(main())
```

Kata kunci `await` menangguhkan coroutine saat menunggu operasi asinkron tanpa memblokir keseluruhan event loop. Dengan mekanisme ini, server dapat melayani beberapa koneksi pada alamat IP dan port Raspberry Pi.

Setelah membuat berkas kode sumber tersebut, jalankan server Python bernama `main.py` melalui terminal.

Menjalankan server python

```
python main.py
```

2.3.2 Source Code ESP32

Sebelum membuat kode, kita perlu mengetahui alamat IP Raspberry Pi yang telah dikonfigurasi sebagai server sebelumnya. Untuk melihat alamat IP Raspberry Pi, jalankan perintah berikut di terminal:

Melihat IP Address dari Raspberry Pi

```
hostname -I
# Atau Anda bisa menggunakan perintah berikut
ip addr
```

Program di atas akan menampilkan alamat IPv4 dan IPv6 Raspberry Pi. Kita hanya memerlukan alamat IPv4 untuk server WebSocket kita, sebagai contoh `192.168.1.100`.

Informasi
Kita akan menggunakan `WebSockets` library dari Markus Setter
https://github.com/Links2004/arduinoWebSockets

Instal library yang diperlukan terlebih dahulu. Selanjutnya lakukan import library seperti biasa pada ESP32.

Library yang digunakan pada ESP32

```
#include <WiFi.h>             // Untuk koneksi ke internet
#include <WebSocketsClient.h> // Koneksi WebSocket
```

Konfigurasi WiFi dan WebSocket

```
// WiFi
const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// WebSocket
const char*    WS_HOST = "192.168.1.100";  // Sesuaikan dengan IP Address Raspberry Pi
const uint16_t WS_PORT = 8765;             // Sesuaikan dengan Port Raspberry Pi

// Untuk membedakan jenis data yang ingin dikirimkan, kita bisa sesuaikan nama path-nya
const char*    WS_PATH = "/esp32";

WebSocketsClient webSocket;

bool webSocketConnected = false;
unsigned long previousSendTime = 0;
const unsigned long SEND_INTERVAL = 1000; // Interval pengiriman data

uint32_t counter = 0;
```

Pastikan IP dan Port yang digunakan di kode ESP32 sama dengan yang ada pada Raspberry Pi.

Fungsi Callback WebSocket

```
void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
    switch (type) {
        case WStype_CONNECTED:
            webSocketConnected = true;

            Serial.println("WebSocket connected.");

            // Optional initial message
            webSocket.sendTXT("{\"status\":\"ESP32-S3 connected\"}");
```

```
            break;

        case WStype_DISCONNECTED:
            webSocketConnected = false;
            Serial.println("WebSocket disconnected.");
            break;

        case WStype_TEXT:
            Serial.print("Message from server: ");
            Serial.write(payload, length);
            Serial.println();
            break;

        case WStype_ERROR:
            Serial.println("WebSocket error.");
            break;

        default:
            break;
    }
}
```

Pada kode di atas, fungsi callback `void webSocketEvent()` digunakan untuk memproses event WebSocket baik memberi maupun menerima pesan.

Koneksi ke Wi-Fi

```
void connectToWiFi() {
    Serial.print("Connecting to Wi-Fi");

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    // Menunggu koneksi Wi-Fi
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("Wi-Fi connected.");

    Serial.print("ESP32 IP address: ");
    Serial.println(WiFi.localIP());
}
```

Sebelum menggunakan WebSocket, kita perlu menghubungkan ESP32 ke jaringan Wi-Fi. Mode `WIFI_STA` memperbolehkan perangkat untuk menjadi sebuah client yang dapat terhubung ke Wi-Fi Access Point (AP) yang tersedia, mirip seperti telepon seluler yang terhubung ke router Wi-Fi. Koneksi dimulai dengan memanggil fungsi `WiFi.begin()` sesuai dengan nama atau SSID Wi-Fi dan password dari koneksi internet yang kita pakai. Pastikan bahwa internet yang kita hubungkan dengan ESP32 sama dengan yang kita hubungkan dengan Raspberry Pi.

Fungsi Setup pada ESP32

```
void setup() {
    Serial.begin(115200);
    delay(1000);

    connectToWiFi();

    webSocket.begin(WS_HOST, WS_PORT, WS_PATH);  // Memulai koneksi WebSocket
    webSocket.onEvent(webSocketEvent);    // Registrasi fungsi callback yang telah dibuat
    webSocket.setReconnectInterval(5000); // Koneksikan ulang bila terputus selama 5 detik
```

```
10
11      // Sambungkan ulang Wi-Fi secara otomatis ketika koneksi terputus
12      WiFi.setAutoReconnect(true);
13  }
```

WebSocket akan dimulai ketika kita berhasil menghubungkan ESP32 ke jaringan. Pada kode di atas, ESP32 akan mencoba terhubung ke Raspberry Pi menggunakan WebSocket sesuai dengan Host, Port, dan Path yang kita atur. Bila koneksi tidak stabil, Wi-Fi dan WebSocket akan mencoba untuk menghubungkan kembali, dengan interval waktu 5 detik untuk WebSocket.

Fungsi Loop pada ESP32

```
1   void loop() {
2       // Memulai loop dari WebSocket
3       webSocket.loop();
4
5       unsigned long currentTime = millis();
6
7       if (
8           webSocketConnected &&
9           currentTime - previousSendTime >= SEND_INTERVAL
10      ) {
11          previousSendTime = currentTime;
12
13          // Letakkan logika pengiriman akan dijalankan di block kodingan berikut
14          // Sebagai contoh, kodingan berikut akan mengirimkan counter tiap detik
15          String message = "{\"counter\":";
16          message += counter;
17          message += "}";
18
19          webSocket.sendTXT(message);
20
21          Serial.print("Sent: ");
22          Serial.println(message);
23
24          counter++;
25      }
26  }
```

Metode `webSocket.loop()` harus dipanggil secara berkala untuk memproses event koneksi WebSocket. Setelah interval pengiriman terpenuhi, metode `webSocket.sendTXT()` mengirimkan pesan teks ke server Raspberry Pi.

```
ESP32 device connected
ESP32 message: {"status":"ESP32-S3 connected"}
ESP32 message: {"counter":0}
ESP32 message: {"counter":1}
ESP32 message: {"counter":2}
ESP32 message: {"counter":3}
ESP32 message: {"counter":4}
ESP32 message: {"counter":5}
```

Gambar 2.4: Output pada Server WebSocket

Contoh di atas adalah output yang akan dikeluarkan oleh server python WebSocket yang telah kita buat sebelumnya.

LAMPIRAN MODUL 2

WebSocket Python Server Source Code

WebSocket Python Server

```python
import asyncio # Supports asynchronous coroutine execution

# WebSocket library
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0" # Listen on all network interfaces
PORT: int = 8765

async def handle_client(websocket: ServerConnection) -> None:
  # Distinguish messages by the WebSocket path
  # The path used in this example is "/esp32"
  path: str = websocket.request.path

  # Select the path to process
  if path == "/esp32":
    print("ESP32 device connected")

    try:
      async for message in websocket:
        # Message from ESP32
        print( f"ESP32 message: {message}" )

        # Reply
        response: str = ( f"ACK from Raspberry Pi: {message}" )
        await websocket.send(response)
    except ConnectionClosed:
      print("ESP32 disconnected")

  # Add other paths as required
  # Default case for an unrecognized path
  else:
  print(f"Unknown path: {path}")

async def main() -> None:
  # Use the configuration defined above
  async with serve(handle_client, HOST, PORT):
  print(f"WebSocket server running " f"on {HOST}:{PORT}")

  # Wait for messages...
  await asyncio.Future()

if __name__ == "__main__":
  asyncio.run(main())
```

Source Code WebSocket dengan ESP32

WebSocket dengan ESP32

```cpp
#include <WiFi.h>
#include <WebSocketsClient.h>

const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

const char*    WS_HOST = "192.168.1.100";
const uint16_t WS_PORT = 8765;
const char*    WS_PATH = "/esp32";

WebSocketsClient webSocket;

```

```
bool webSocketConnected = false;
unsigned long previousSendTime = 0;
const unsigned long SEND_INTERVAL = 1000;
uint32_t counter = 0;

void webSocketEvent(WStype_t type, uint8_t* payload, size_t length) {
  switch (type) {
    case WStype_CONNECTED:
      webSocketConnected = true;
      Serial.println("WebSocket connected.");
      webSocket.sendTXT("{\"status\":\"ESP32 connected\"}");
      break;

    case WStype_DISCONNECTED:
      webSocketConnected = false;
      Serial.println("WebSocket disconnected.");
      break;

    case WStype_TEXT:
      Serial.print("Message from server: ");
      Serial.write(payload, length);
      Serial.println();
      break;

    case WStype_ERROR:
      Serial.println("WebSocket error.");
      break;

    default:
      break;
  }
}

void connectToWiFi() {
  Serial.print("Connecting to Wi-Fi");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connected.");
  Serial.print("ESP32 IP address: ");
  Serial.println(WiFi.localIP());
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  connectToWiFi();

  webSocket.begin(WS_HOST, WS_PORT, WS_PATH);
  webSocket.onEvent(webSocketEvent);
  webSocket.setReconnectInterval(5000);
  WiFi.setAutoReconnect(true);
}

void loop() {
  webSocket.loop();

  unsigned long currentTime = millis();
  if (
    webSocketConnected &&
```

```
      currentTime - previousSendTime >= SEND_INTERVAL
    ) {
      previousSendTime = currentTime;

      String message = "{\"counter\":";
      message += counter;
      message += "}";

      webSocket.sendTXT(message);
      Serial.print("Sent: ");
      Serial.println(message);
      counter++;
    }
  }
```